

Capitolo 10 - Memoria Virtuale

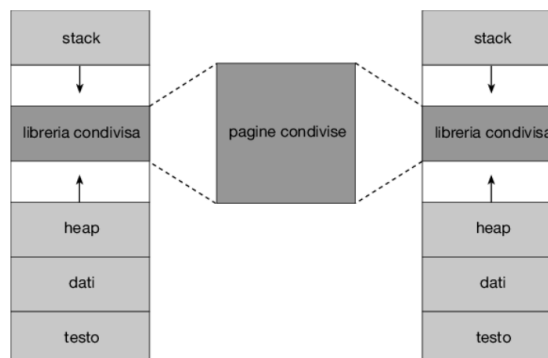
Status Done

Memoria Virtuale

La memoria virtuale è una tecnica di gestione della memoria che permette di eseguire processi che non sono interamente contenuti nella memoria fisica.

Concetti chiave e vantaggi:

- **Programmi più grandi della memoria fisica:** il vantaggio principale è che i programmi possono avere dimensioni maggiori della memoria fisica disponibile.
- **Non è necessario l'intero programma:** spesso, un programma contiene porzioni di codice (come le routine di gestione degli errori o porzioni di codice meno usate) che non sono necessarie per l'intera esecuzione.
- **Separazione logica/fisica:** la memoria virtuale astrae la memoria centrale in un array di memorizzazione molto grande e uniforme, separando la memoria logica (come vista dall'utente/programmatore) da quella fisica.
- **Spazio degli indirizzi virtuali:** l'espressione si riferisce alla collocazione dei processi in memoria dal punto di vista logico, dove un processo tipicamente inizia a un indirizzo logico 0 e occupa uno spazio contiguo.
 - **Condivisione di risorse:** la memoria virtuale consente ai processi di condividere facilmente:
 - **Librerie di sistema:** mappando l'oggetto di memoria condiviso nello spazio degli indirizzi virtuali, più processi possono utilizzare le stesse pagine fisiche in sola lettura.
 - **Memoria condivisa:** utilizzata per la comunicazione tra processi, mappando la stessa regione di memoria fisica nello spazio di indirizzi virtuali dei processi comunicanti.



Paginazione su richiesta

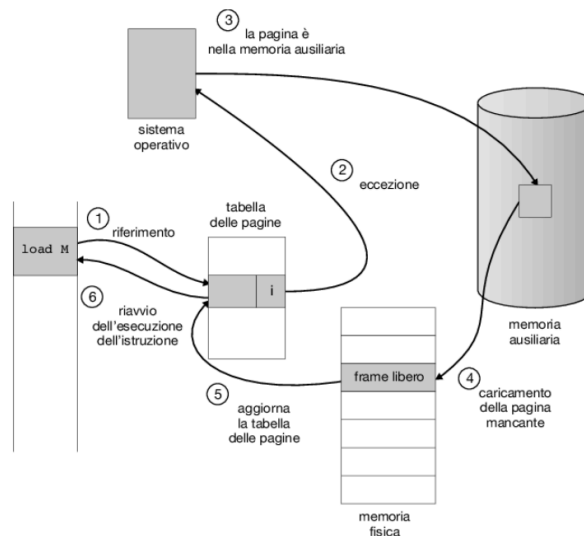
La paginazione su richiesta è la tecnica comunemente adottata dai sistemi con memoria virtuale e consiste nel **caricare le pagine in memoria fisica solo nel momento in cui sono effettivamente necessarie** (richieste) durante l'esecuzione del programma. Le pagine mai utilizzate non vengono mai caricate.

Implementazione di base

Per distinguere le pagine presenti in memoria da quelle non presenti, si usa una variazione del meccanismo di paginazione:

- **Bit di validità/Non Validità:** una pagina non in memoria (sul disco) è contrassegnata come **non valida** (bit di validità impostato a 0 o con un valore speciale nei bit di protezione)
- **Page Fault (Pagina Mancante):** se un processo tenta di accedere a una pagina non valida, l'hardware di paginazione genera una **trap (eccezione)** per il SO, nota come *page fault*.
- **Gestione del page fault:** la procedura per gestire un *page fault* comporta i seguenti passaggi principali:
 1. **Verifica:** controllare se l'accesso alla memoria è valido o non valido. Se non valido, il processo viene terminato.
 2. **Caricamento:** se valido ma assente, si individua la pagina sul disco (nella memoria secondaria/area di swap).
 3. **Frame libero:** si cerca un frame (blocco di memoria fisica) libero. Se non ce ne sono, si usa un **algoritmo di sostituzione** per scegliere una "pagina vittima" da sostituire.

4. **Trasferimento:** la pagina richiesta viene caricata nel frame libero. Si aggiorna la tabella delle pagine del processo per indicare la pagina come valida.
5. **Riavvio:** l'istruzione che ha causato il *page fault* viene riavviata.



Attesa indefinita

Se un processo incorre in un *page fault*, deve attendere il completamento dell'operazione di I/O, che può essere molto lenta. Durante questo tempo, il processo entra in uno stato bloccato.

- **Liste di frame:** per mantenere l'uso della CPU e massimizzare le prestazioni, il SO tiene traccia dei frame fisici non utilizzati attraverso una **lista di frame liberi**. Il SO utilizza la CPU per eseguire altri processi mentre la pagina richiesta viene trasferita dal disco.

Prestazioni della paginazione su richiesta

La paginazione su richiesta incide notevolmente sulle prestazioni, a causa dell'overhead del page fault, che include l'I/O su disco.

- **Tempo di accesso effettivo (TAE):** il tempo impiegato per un accesso alla memoria è calcolato considerando la probabilità di un *page fault* (p) e il tempo necessario per gestirlo.

$$TAE = (1 - p) \times TA + p \times TP$$

Dove:

- TA : tempo di accesso (alla memoria fisica, es. 10ns).
- TP : tempo di page fault (tempo necessario per gestire il fault, inclusa l'I/O da disco, es. 25ms).
- **Impatto critico:** poiché il tempo di gestione del *page fault* (TP) è milioni di volte maggiore del tempo di accesso alla memoria (TA), anche un tasso di page fault molto basso (come $p = 1/1000$) degrada significativamente le prestazioni.

Copiatura su scrittura (Copy-on-Write)

La copiatura su scrittura è una tecnica fondamentale utilizzata per ottimizzare la creazione di processi, in particolare dopo una chiamata di sistema `fork()` (come in Linux, Unix), che altrimenti comporterebbe la duplicazione immediata e inefficiente dell'intero spazio di indirizzi del processo genitore.

Meccanismo di base

L'obiettivo della `fork()` è creare un processo figlio come duplicato esatto del genitore. Tradizionalmente, questo richiedeva la copia immediata di tutte le pagine del genitore per il figlio, un'operazione che spreca tempo ed energia se il figlio esegue immediatamente la chiamata `exec()` (per caricare un nuovo programma) subito dopo.

La CoW risolve questo problema con il seguente meccanismo:

1. **Condivisione iniziale:** invece di copiare immediatamente tutte le pagine del genitore, **genitore e figlio condividono le stesse pagine in memoria fisica**.
2. **Protezione:** queste pagine condivise vengono contrassegnate come di **sola lettura** (read-only).

3. **Copia al bisogno:** solo quando uno dei due processi (genitore o figlio) tenta di scrivere su una di queste pagine condivise, si verifica un *page fault*.
4. **Gestione del fault:** il SO intercetta il *page fault*, crea una **copia esatta della pagina** in un nuovo *frame* e la mappa nello spazio di indirizzi del processo che ha tentato la scrittura. A questo punto, la copia locale diventa scrivibile (lettura-scrittura) per quel processo, mentre l'originale resta in uso (o non modificato) dall'altro processo.

Vantaggi

- **Efficienza nella creazione del processo:** la CoW garantisce la **celere generazione dei processi** riducendo al minimo il numero di nuove pagine allocate al processo appena creato, dato che la copia delle pagine avviene solo se e quando necessario.
- **Risparmio di risorse:** si evita il sovraccarico inutile di copiare pagine che potrebbero non essere mai modificate, specialmente quando il processo figlio esegue subito un `exec()`.

Variante `vfork()` (Virtual Memory Fork)

Alcune versioni di Unix (tra cui Linux, macOS, e Unix BSD) offrono una variante della `fork()` chiamata `vfork()` (per *virtual memory fork*).

- **Condivisione e sospensione:** con la `vfork()`, il processo genitore viene sospeso e il processo figlio usa lo spazio di indirizzi del genitore (senza la protezione CoW).
- **Pericolo e utilizzo:** questa chiamata richiede estrema cautela: se il figlio modifica lo spazio di indirizzi, le modifiche saranno immediatamente visibili al genitore quando riprenderà. La `vfork()` è adatta unicamente quando il processo figlio esegue una `exec()` immediatamente dopo la sua creazione.

Sostituzione delle pagine

Quando si aumenta il grado di multiprogrammazione (cioè il numero di processi in esecuzione contemporaneamente) in un sistema di paginazione su richiesta, la memoria viene **sovrassegnata**. Se i processi tentano di utilizzare più memoria di quanta ne sia disponibile, il sistema può incorrere in un'eccessiva attività di paginazione. Per gestire questa situazione e consentire l'esecuzione di nuovi processi quando tutti i frame sono occupati, è necessario implementare la **sostituzione delle pagine**.

Sostituzione di pagina

L'obiettivo della sostituzione delle pagine è: **se nessun frame è libero, ne viene liberato uno attualmente inutilizzato**. Il frame liberato verrà poi utilizzato per ospitare la pagina che ha causato il *page fault*. La procedura di servizio dell'eccezione di *page fault* viene quindi modificata per includere la sostituzione:

1. Si individua la posizione su disco della pagina richiesta.
2. Si cerca un frame libero. Se non c'è, si sceglie una pagina vittima con l'algoritmo di sostituzione.
3. **Gestione del Dirty Bit (Dirty Bit):**
 - Prima di liberare il frame della vittima, il SO controlla il bit di modifica associato alla pagina nella tabella delle pagine.
 - **Se il bit di modifica è 1 (Modificata):** la pagina è stata scritta e non è più identica alla sua copia sul disco. È necessario riscriverla sul disco prima di liberare il frame (operazione di I/O aggiuntiva).
 - **Se il bit di modifica è 0 (Non Modificata):** la pagina è identica alla copia sul disco e il frame può essere immediatamente liberato e riutilizzato, risparmiando un'operazione di I/O.
4. Si aggiornano le tabelle della vittima e si carica la nuova pagina richiesta nel frame appena liberato.

Il problema principale della paginazione su richiesta è lo sviluppo di un algoritmo di sostituzione che garantisca un **basso tasso di page fault**, dato che l'I/O su disco è molto oneroso.

Algoritmo FIFO

Il più semplice algoritmo di sostituzione associa a ogni pagina l'istante di tempo in cui è stata portata in memoria.

- **Regola:** si sostituisce la pagina che è **presente in memoria da più tempo**.
- **Implementazione:** si utilizza una coda FIFO per tenere traccia delle pagine; la pagina in testa è quella da sostituire.
- **Costo/Efficienza:**
 - **Implementazione:** molto semplice e basso overhead.
 - **Prestazioni:** spesso non efficiente. Una pagina caricata molto tempo fa potrebbe in realtà essere una **pagina frequentemente usata** (es. una pagina contenente un ciclo di programma) e la sua rimozione causerebbe subito un *page fault*.

- **Criticità: Anomalia di Belady**

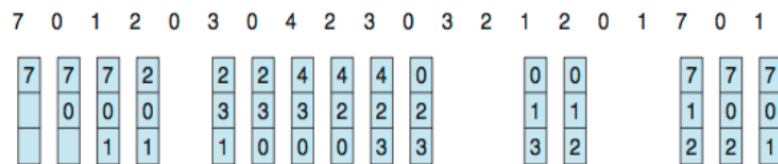
- FIFO è l'unico algoritmo comune a soffrire di questa anomalia (o anomalia della pila).
- L'anomalia si verifica quando un **aumento del numero di frame** disponibili per un processo (es. si passa da 3 a 4 frame) comporta, per una specifica sequenza di riferimenti, un aumento del numero di page fault.

- ▼ **Esempio pratico**

- **Stringa di riferimento:** è la sequenza di pagine a cui il processo tenta di accedere, nell'ordine: **7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1**.
- **Frame:** il processo ha a disposizione **3 frame** (spazi fisici di memoria). Ciò significa che solo 3 pagine possono trovarsi contemporaneamente in memoria in un dato momento.

Per tracciare l'età delle pagine, si utilizza semplicemente una coda FIFO. La pagina in testa alla coda è la vittima della sostituzione.

Risultato del caso con 3 frame: l'esempio porta a 15 page faults.



Algoritmo ottimale (OPT e MIN)

Questo algoritmo serve come punto di riferimento teorico, fornendo il limite inferiore assoluto del tasso di page fault.

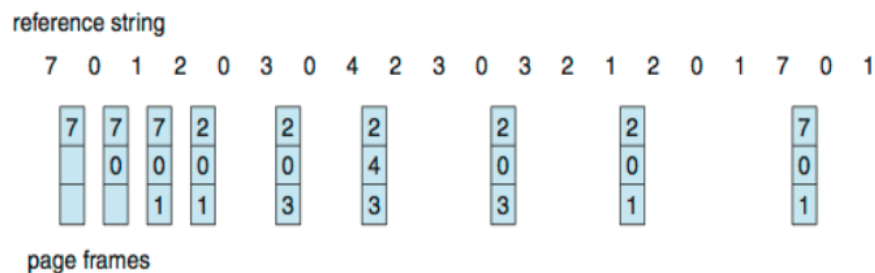
- **Regola operativa:** sostituire la pagina che non verrà usata per il periodo di tempo più lungo nel futuro della sequenza di riferimenti.
- **Meccanismo:** per ogni pagina in memoria, si calcola l'intervallo di tempo prima del suo prossimo utilizzo. Si sceglie come vittima la pagina con l'intervallo più lungo.
- **Costo/Efficienza:**
 - **Prestazioni:** assicura il tasso minimo di page fault per un dato numero di frame. Non è soggetto all'anomalia di Belady.
 - **Realizzabilità:** è difficile da realizzare (impraticabile) perché richiede la conoscenza futura della sequenza di riferimenti alla memoria. Per questo motivo, è utilizzato quasi esclusivamente per studi comparativi.

- ▼ **Esempio pratico**

L'algoritmo utilizza i seguenti elementi:

- **Stringa di Riferimento:** 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1
- **Frame Disponibili:** 3

L'applicazione dell'algoritmo ottimale alla sequenza di riferimenti fornita risulterebbe in 9 page faults.

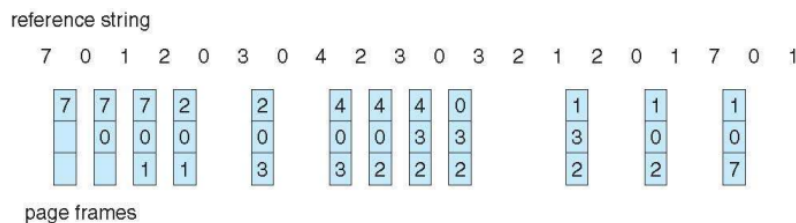


Algoritmo LRU (Least Recently Used)

LRU è l'algoritmo che cerca di approssimare il comportamento di OPT, basandosi sul principio che il passato recente è un buon predittore del futuro.

- **Regola operativa:** sostituire la pagina che è stata **usata meno recentemente** nel passato.
- **Meccanismo:** l'obiettivo è identificare la pagina con il tempo trascorso più lungo dal suo ultimo riferimento.

- **Implementazione con contatori:** richiederebbe l'associazione di un contatore di tempo logico (o *timestamp*) ad ogni pagina, da aggiornare ad ogni riferimento. La pagina vittima è quella con il valore di tempo più basso.
- **Implementazione con stack:** mantenere uno stack delle pagine. Ad ogni riferimento, la pagina viene spostata in cima. La pagina vittima è sempre quella in fondo allo stack.
- **Criticità di implementazione (costo HW):**
 - Entrambi i meccanismi richiedono un notevole e costoso supporto HW aggiuntivo per tenere traccia dell'ordine di tutti gli accessi alla memoria, con un alto overhead.
- **Vantaggi:** non è soggetto all'anomalia di Belady. Appartiene alla classe degli **algoritmi a stack**.
- ▼ **Esempio pratico**
 - **Stringa di Riferimento (Reference String):** 7, 0, 1, 2, 0, 3, 0, 4, 2, 3, 0, 3, 2, 1, 2, 0, 1, 7, 0, 1.
 - **Frame Disponibili:** 3.



L'algoritmo produce 12 page faults.

Algoritmi di approssimazione LRU

L'algoritmo LRU è efficace ma costoso da implementare a causa dell'overhead richiesto per tracciare l'ordine esatto di utilizzo di ogni pagina. Gli algoritmi di approssimazione LRU offrono un compromesso, utilizzando funzionalità HW più semplici per ottenere risultati comparabili.

Algoritmo aggiuntivo di riferimento (Additional-Reference-Bit Algorithm)

Questo algoritmo utilizza la funzione HW di un **bit di riferimento** per registrare l'uso recente delle pagine:

- **Bit di riferimento (Reference Bit):** ogni voce della tabella delle pagine possiede un bit di riferimento.
- **Funzione HW:** l'HW imposta automaticamente il bit a 1 ogni volta che si verifica un riferimento (lettura o scrittura) alla pagina.
- **Meccanismo:**
 1. A intervalli di tempo regolari (es. ogni 100 millisecondi), un processo in background (chiamato timer o *age counter*) scansa tutte le pagine in memoria.
 2. Per ogni pagina, il processo **legge il valore del bit di riferimento** e lo accoda al registro dello storico di utilizzo della pagina (un contatore di 8 bit).
 3. Il bit di riferimento viene quindi **azzerato**.
- **Storico di utilizzo:** il registro di 8 bit memorizza la cronologia di utilizzo; ad esempio, se il valore è 00000001, significa che la pagina è stata usata nell'intervallo di tempo più recente. La pagina con il valore più basso viene scelta come vittima, approssimando così l'LRU.

Algoritmo Seconda Chance (Second-Chance Algorithm)

Questo algoritmo, noto anche come algoritmo **A Orologio (Clock)**, è una variante dell'algoritmo FIFO che utilizza il bit di riferimento per prendere decisioni più intelligenti, con un overhead molto inferiore a LRU.

- **Meccanismo FIFO Modificato:** quando è necessaria una sostituzione, l'algoritmo (che utilizza una coda circolare, come un quadrante di orologio) ispeziona la pagina più vecchia (la testa della coda).
- **Regola della seconda chance:**
 1. **Se il bit di riferimento è 0:** la pagina non è stata usata di recente. È considerata una vittima e viene sostituita (segue FIFO).
 2. **Se il bit di riferimento è 1:** la pagina è stata usata di recente. Le viene data una seconda chance:
 - Il bit di riferimento viene azzerato.
 - L'istante di arrivo della pagina viene aggiornato all'ora corrente.
 - Il puntatore dell'orologio avanza alla pagina successiva.

- **Vantaggio:** una pagina non viene sostituita finché tutte le altre pagine non hanno avuto la loro seconda chance. Le pagine usate di frequente mantengono il loro bit a 1 e non vengono mai sostituite, a differenza di FIFO.
- **Implementazione:** si utilizza una **coda circolare** con un puntatore (la **lancetta dell'orologio**) che indica la prossima vittima potenziale.

Algoritmo LRU Aumentato (Enhanced Second-Chance Algorithm)

Questo algoritmo migliora l'algoritmo Seconda Chance utilizzando una combinazione del **Bit di Riferimento** e del **Bit di Modifica** (Dirty Bit) per creare quattro classi di priorità per la sostituzione:

- **Classe (0, 0): bit di riferimento = 0, bit di modifica = 0.** Pagina **non usata** e **non modificata**. È la vittima ideale, poiché non è stata usata e non richiede I/O su disco (due operazioni salvate).
- **Classe (0, 1): bit di riferimento = 0, bit di modifica = 1.** Pagina **non usata** ma **modificata**. Non è stata usata di recente, ma richiede I/O prima di essere liberata.
- **Classe (1, 0): bit di riferimento = 1, bit di modifica = 0.** Pagina **usata di recente** ma **non modificata**. Potrebbe essere usata di nuovo, ma non richiede I/O se viene sostituita.
- **Classe (1, 1): bit di riferimento = 1, bit di modifica = 1.** Pagina **usata** di recente e **modificata**. È la peggiore vittima potenziale: è stata usata di recente e richiede I/O.

L'algoritmo scansiona i frame in un ordine di priorità, scegliendo la vittima dalla classe con indice più basso (inizia cercando la classe (0, 0)).

Sostituzione delle pagine basata su conteggio

Un altro approccio alla sostituzione delle pagine consiste nell'associare a ciascuna pagina un contatore del numero di riferimenti che le sono stati fatti. Questo contatore può essere utilizzato per implementare due algoritmi principali.

Algoritmo LFU (Least Frequently Used)

- **Regola:** sostituire la pagina con il **conteggio dei riferimenti più basso**.
- **Logica:** si basa sul presupposto che una pagina usata attivamente avrà un conteggio di riferimenti elevato.
- **Problema:** LFU non tiene conto del tempo. Una pagina che è stata usata intensamente durante la fase iniziale di un processo (quindi ha un conteggio alto) ma che non viene più usata in seguito, rimarrebbe in memoria a discapito di pagine più nuove e attive.
- **Possibile soluzione:** per mitigare questo problema, si può utilizzare un **conteggio con peso esponenziale decrescente**, spostando a destra di un bit (dividendo per 2) valori dei contatori a intervalli regolari. Questo dà meno peso ai riferimenti più vecchi.

Algoritmo MFU (Most Frequently Used)

- **Regola:** sostituire la pagina con il **conteggio dei riferimenti più alto**.
- **Logica:** si basa sul presupposto che la pagina con il conteggio più basso sia stata appena caricata e che sia ancora in attesa di essere usata.

Nota:

Sia LFU che MFU, nella loro forma pura, non sono molto popolari a causa del loro alto costo di gestione dei contatori e della difficoltà di riflettere accuratamente l'uso corrente della memoria.

Algoritmi con buffering delle pagine

Vengono utilizzate procedure di buffering per ottimizzare la gestione delle pagine e l'efficienza, integrando qualsiasi algoritmo di sostituzione.

Gruppo di frame liberi (Pool of Free Frames)

- I sistemi mantengono un gruppo di frame liberi.
- Quando si verifica un page fault, la pagina richiesta viene trasferita immediatamente in un frame libero del gruppo.
- La pagina vittima viene scelta, ma la sua scrittura su disco è differita.
- Il processo può ricominciare al più presto, senza attendere l'I/O della vittima.

Scrittura proattiva e pagine modificate

- Il sistema mantiene una lista delle pagine modificate (*dirty pages*).

- Quando il dispositivo di paginazione è inattivo, le pagine modificate vengono scritte proattivamente sul disco e il loro bit di modifica viene resettato.
- **Vantaggio:** aumenta la probabilità che, al momento della selezione per la sostituzione, la pagina sia già pulita, eliminando la necessità di scriverla su disco in quel momento.

Caching dei frame rilasciati

- Si usa il pool of free frames per la caching temporanea delle pagine appena rimosse.
- Il sistema ricorda quale pagina era contenuta in ciascun frame del pool.
- Se un *page fault* richiede una pagina che è ancora nel pool (e non riusata), non è necessario alcun I/O.
- **Obiettivo:** ridurre il costo dell'I/O dovuto all'eventuale scelta errata dell'algoritmo di sostituzione.

Applicazione e sostituzione della pagina

Gli algoritmi di gestione della memoria, sebbene universali, possono essere inefficienti per applicazioni specializzate (es. database) che hanno pattern di accesso unici.

Inefficienza degli algoritmi generici

- **Doppio buffering:** se un'applicazione adotta il proprio buffering per l'I/O e lo fa anche il SO, la memoria necessaria viene inutilmente raddoppiata.
- **Algoritmi inadeguati:** le applicazioni che effettuano lunghe letture sequenziali (es. data warehouse):
 - LRU eliminerebbe le pagine più vecchie.
 - In queste circostanze, MFU sarebbe più efficiente, poiché la riletture delle pagine vecchie è attesa.

Soluzione: I/O di basso livello (Raw I/O)

Per risolvere questi problemi e permettere alle applicazioni di ottimizzare l'I/O, alcuni sistemi offrono un accesso diretto al dispositivo.

- **Raw Disk (disco di basso livello):** alcuni programmi possono utilizzare una partizione del disco come un array sequenziale di blocchi logici, senza ricorrere alle strutture del file system.
- **Raw I/O (I/O di basso livello):** questa tecnica bypass i servizi del file system, inclusi:
 - la paginazione su richiesta dell'I/O su file;
 - il locking del file;
 - il prefetching e l'allocazione dello spazio;
 - la gestione dei nomi dei file e delle directory.

Sebbene l'I/O di basso livello sia più efficiente per il *buffering* interno di alcune applicazioni specializzate, la maggior parte delle applicazioni ha una resa migliore operando con i servizi regolari del file system.

Allocazione dei frame

Il problema dell'allocazione dei frame consiste nello stabilire un criterio per la distribuzione della memoria fisica libera tra i diversi processi utente in esecuzione.

Esempio:

- Se un sistema ha 128 frame totali, il SO ne occupa 35, lasciandone 93 per i processi utente.
- In condizioni di paginazione su richiesta ("pura"), tutti i 93 frame sono inizialmente nella lista dei frame liberi.
- Quando un processo utente inizia a eseguire, i suoi primi page fault vengono soddisfatti attingendo da questa lista.
- Una volta esaurita la lista, l'algoritmo di sostituzione delle pagine entra in gioco per scegliere quale pagina sostituire per far posto alla successiva.

Varianti della strategia:

- Il SO può richiedere e gestire parte dei frame liberi per le proprie strutture dati, restituendoli per la paginazione utente quando inutilizzati.
- È comune riservare alcuni frame liberi (es. 3 frame) come buffer per gestire immediatamente un page fault e avviare il trasferimento della pagina richiesta dal disco, permettendo al processo utente di continuare a scegliere la pagina vittima mentre il trasferimento è in corso.

- La strategia di base rimane: assegnare al processo utente i frame liberi disponibili.

Numero minimo di frame

Le strategie di allocazione sono soggette a due vincoli principali: non allocare più frame di quelli disponibili e non allocare meno di un numero minimo di frame.

Motivazione del vincolo minimo:

1. **Prestazioni:** un numero insufficiente di frame aumenta drasticamente il tasso di page fault, rallentando l'esecuzione.
2. **Architettura delle istruzioni (Instruction Set):** l'esecuzione di una singola istruzione richiede che tutte le pagine a cui fa riferimento siano presenti in memoria. Se si verifica un page fault prima che l'istruzione sia completata, questa deve essere riavviata.

Requisiti architetturali:

- Se un'istruzione fa riferimento a un solo indirizzo di memoria, sono necessari almeno due frame: uno per l'istruzione stessa e uno per il riferimento alla memoria.
- Se è ammesso l'indirizzamento indiretto a un livello, potrebbero essere necessari almeno tre frame (uno per l'istruzione, uno per il primo riferimento e uno per il riferimento indiretto).
- Se un'istruzione (es. `move`) può estendersi su due pagine e i suoi due operandi sono riferimenti indiretti, l'istruzione potrebbe richiedere fino a sei frame.

Determinazione dei limiti:

- Il **numero minimo di frame** è definito dall'**architettura** del calcolatore.
- Il **numero massimo di frame** è definito dalla quantità di memoria fisica disponibile.

Algoritmi di allocazione

Gli algoritmi di allocazione definiscono la metodologia con cui i frame disponibili m vengono suddivisi tra gli n processi.

1. Allocazione Uniforme (Equal Allocation):

- Si assegna a ciascun processo una parte uguale dei frame disponibili: m/n frame per ogni processo.
- Es. con 93 frame e 5 processi, ciascuno riceve 18 frame. I frame rimanenti possono essere usati come buffer liberi.

2. Allocazione proporzionale (Proportional Allocation):

- La memoria viene assegnata a ciascun processo in base alla sua **dimensione virtuale** (s_i).
- Il numero di frame a_i assegnati al processo P_i è calcolato approssimativamente come:

$$a_i = \frac{s_i}{S} \times m$$

Dove S è la somma delle dimensioni virtuali di tutti i processi ($\sum s_i$).

- Questo approccio garantisce che i processi più grandi ricevano più memoria, riflettendo le loro maggiori necessità.

Variazioni dinamiche e priorità:

- In entrambi gli schemi, l'allocazione per processo varia in funzione del livello di multiprogrammazione: se nuovi processi arrivano, i frame di ciascuno diminuiscono; se i processi terminano, i frame vengono ridistribuiti.
- Per supportare la priorità, è possibile modificare lo schema proporzionale in modo che il rapporto di allocazione non sia basato solo sulle dimensioni, ma sulle priorità dei processi o una combinazione di dimensioni e priorità.

Allocazione globale e allocazione locale

Queste due strategie definiscono come avviene la sostituzione delle pagine quando si verifica un *page fault* e come essa influisce sull'allocazione dei frame tra i processi.

Strategia	Descrizione	Effetto sull'Allocazione dei Frame	Vantaggi / Svantaggi
Sostituzione Globale	Un processo può scegliere un frame vittima dall'insieme di tutti i frame in memoria , anche quelli allocati ad altri processi.	L'insieme di frame allocati a un processo può aumentare o diminuire dinamicamente.	Vantaggio: Maggiore produttività (<i>throughput</i>) del sistema. Svantaggio: Un processo non ha controllo sul proprio tasso di <i>page fault</i> (è influenzato da altri processi).

Sostituzione Locale	Un processo può scegliere un frame vittima soltanto dal proprio insieme di frame allocati .	Il numero di frame allocati a un processo non cambia .	Vantaggio: Le prestazioni di un processo sono coerenti e prevedibili (dipendono solo dal suo comportamento). Svantaggio: Può penalizzare un processo impedendogli di acquisire memoria meno utilizzata da altri.
----------------------------	--	---	---

Strategia globale implementata con i "Reaper" (Mietitrici):

- È una possibile implementazione della sostituzione globale, che mira a mantenere una lista di frame liberi sempre rifornita.
- Quando i frame liberi scendono al di sotto di una soglia minima, una routine del kernel (chiamata reaper) si attiva in background.
- Il reaper recupera pagine dai processi (escluso il kernel), utilizzando un algoritmo di sostituzione (spesso un'approssimazione di LRU), e le aggiunge alla lista libera.
- Il reaper si sospende quando la memoria libera raggiunge una soglia massima.
- In casi di memoria estremamente bassa, sistemi come Linux possono attivare l'OOM Killer (Out-Of-Memory killer), che seleziona un processo con il punteggio OOM più alto (basato sulla percentuale di memoria utilizzata) da terminare, al fine di liberare memoria.

Accesso non uniforme alla memoria (NUMA)

In passato si assumeva che l'accesso a tutte le aree della memoria fisica fosse uniforme. L'architettura NUMA introduce una complessità.

Architettura NUMA:

- Sistemi multiprocessore in cui diverse CPU hanno una propria memoria locale (più veloce) e possono accedere alla memoria degli altri nodi (remota e più lenta).
- L'accesso alla memoria è non uniforme: la latenza di accesso varia in base alla posizione fisica della memoria rispetto alla CPU.

Allocazione consapevole di NUMA:

- Le decisioni di allocazione della memoria influiscono drasticamente sulle prestazioni.
- L'obiettivo è allocare i frame di memoria "il più vicino possibile" alla CPU su cui è in esecuzione il processo, intendendo con ciò la latenza minima (tipicamente sulla stessa scheda).
- Quando si verifica un page fault, il gestore della memoria virtuale assegna un frame dal nodo di memoria più vicino alla CPU che ha generato il fault.
- Per supportare questa politica, lo scheduler deve mantenere traccia dell'ultima CPU su cui è stato eseguito un processo/thread.
- Sistemi come Linux e Solaris implementano meccanismi (domini di scheduling, lgroup) per tentare di schedulare i thread e allocare la loro memoria all'interno dello stesso nodo o del nodo più vicino, minimizzando la latenza di accesso alla memoria e massimizzando il tasso di cache hit.

Thrashing

Il **thrashing** è una condizione che si verifica quando un processo non dispone di un numero di frame di memoria "sufficiente" a contenere il suo *working set* (l'insieme di pagine utilizzate attivamente).

Meccanismo:

1. Il processo incorre rapidamente in un page fault (pagina mancante).
2. Poiché tutte le sue pagine sono attive e necessarie (sono nel working set), il processo deve sostituire una pagina che sarà immediatamente richiesta.
3. Di conseguenza, si verificano continui page fault perché le pagine sostituite vengono subito riportate in memoria.
4. **Effetto:** il processo spende più tempo nell'attività di paginazione (lettura/scrittura di pagine dal disco) che nell'esecuzione vera e propria, causando gravi problemi di prestazioni.

Cause del thrashing

Il thrashing può essere innescato da una gestione errata del grado di multiprogrammazione (il numero di processi eseguiti contemporaneamente):

Scenario classico (sostituzione globale):

1. **Basso utilizzo della CPU:** il SO, rilevando un basso utilizzo della CPU (indice di inefficienza), tenta di aumentarlo introducendo un nuovo processo (aumenta il grado di multiprogrammazione).

2. **Richiesta di frame:** un processo esistente entra in una nuova fase di esecuzione e richiede più frame, generando page fault.
3. **Contagio:** usando un algoritmo di sostituzione delle pagine globale, questo processo sottrae frame ad altri processi.
4. **Effetto domino:** gli altri processi, avendo perso pagine necessarie, subiscono anch'essi page fault e si mettono in coda per il dispositivo di paginazione, portando la coda dei processi pronti (*ready queue*) a svuotarsi.
5. **Crollo:** l'utilizzo della CPU diminuisce drasticamente, ma lo scheduler interpreta erroneamente questo calo come una necessità di aumentare ancora il grado di multiprogrammazione, aggravando la situazione.
6. **Risultato:** si ha un **crollo della produttività** dove i processi non svolgono lavoro utile, spendendo tutto il tempo in paginazione.

Modello di località (per la prevenzione)

- Per evitare il thrashing, è necessario fornire a un processo tutti i frame di cui necessita.
- L'approccio del working set si basa sul **modello di località**: un processo, durante l'esecuzione, si sposta di località in località, dove una località è un insieme di pagine usate attivamente insieme.
- L'obiettivo è allocare frame sufficienti a contenere le pagine attive di un processo.

Modello del working set

Il modello del working set è basato sull'ipotesi di località e serve come approccio per il controllo del thrashing.

- **Definizione del working set:** si utilizza un parametro Δ (finestra del working set) per definire l'insieme di pagine cui il processo ha fatto riferimento nei più recenti Δ riferimenti. Questo insieme è una approssimazione della località del programma.
 - Una pagina è nel working set se è stata usata attivamente. Se non è più usata, esce dal working set dopo Δ unità di tempo dal suo ultimo riferimento.
- **Dimensione totale richiesta (D):** la dimensione del working set wss_i per ogni processo P_i viene calcolata, e la richiesta totale di frame è:

$$D = \sum wss_i$$

- **Prevenzione del thrashing:**
 - Se la richiesta totale D è maggiore del numero totale di frame disponibili m ($D > m$), il sistema deve sospendere un processo per liberare i suoi frame, evitando il thrashing.
 - Questa strategia mantiene il grado di multiprogrammazione al livello massimo possibile che può essere sostenuto dalla memoria fisica, ottimizzando l'utilizzo della CPU.
- **Implementazione (Approssimazione):** tracciare continuamente il working set è complesso. Si può usare il timer a intervalli fissi insieme al bit di riferimento (impostato a 1 a ogni riferimento e periodicamente azzerato) per stabilire se una pagina è stata usata in un recente intervallo di tempo.

Frequenza dei page fault (Page Fault Frequency, PFF)

La strategia PFF è un metodo più diretto per controllare il thrashing rispetto al modello del working set.

- **Concetto:** si basa sul presupposto che una frequenza eccessiva di page fault (alta PFF) indica che il processo necessita di più frame. Al contrario, una frequenza molto bassa (bassa PFF) indica che al processo potrebbero essere stati assegnati troppi frame.
- **Meccanismo:** vengono fissati un limite inferiore e un limite superiore per la frequenza desiderata dei page fault.
 - Se la PFF effettiva supera il limite superiore, si alloca un altro frame al processo.
 - Se la PFF effettiva scende sotto il limite inferiore, si sottrae un frame al processo.
- **Thrashing e PFF:** se la PFF aumenta e non ci sono frame disponibili da allocare, il sistema deve selezionare un processo e spostarlo nel backing store (swapping), e i frame così liberati vengono distribuiti ai processi con elevata PFF.

Allocazione di memoria del kernel

Il kernel gestisce l'allocazione di memoria per i processi utente prendendo le pagine dalla lista dei frame disponibili. Tuttavia, i processi in modalità kernel (il kernel stesso) utilizzano una riserva di memoria libera differente da quella usata per i processi utente, principalmente per due motivi:

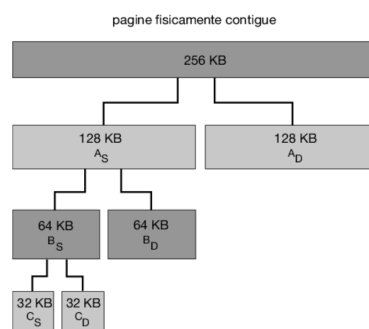
1. **Frammentazione e dimensioni variabili:** il kernel richiede memoria per strutture dati con dimensioni variabili, spesso minori di una singola pagina. Allocando un intero frame per piccole strutture si otterrebbe una notevole frammentazione interna, e questo spreco è critico poiché il codice e i dati del kernel in molti SO non sono paginabili.

2. **Contiguità fisica per l'I/O:** le pagine allocate ai processi utente non devono essere fisicamente contigue. Alcuni dispositivi I/O, però, interagiscono direttamente con la memoria fisica (senza l'interfaccia della memoria virtuale) e possono richiedere lotti di memoria che risiedano in pagine fisicamente contigue.

Sistema buddy

È un metodo di allocazione di memoria basato su segmenti di pagine fisicamente contigue.

- **Allocatore a potenze di 2:** alloca la memoria in unità la cui dimensione è pari a una potenza di 2 (es. 4 KB, 8 KB, 16 KB). Qualsiasi richiesta di memoria viene arrotondata per eccesso alla successiva potenza di 2.
- **Meccanismo (Suddivisione):** un segmento iniziale viene ripetutamente suddiviso in due buddy (compagni) di dimensione dimezzata. La suddivisione prosegue finché non si ottiene un segmento di dimensione sufficiente a soddisfare la richiesta (es. una richiesta di 21 KB viene soddisfatta con un segmento di 32 KB).
- **Fusione (Coalescing):** quando un segmento viene rilasciato dal kernel, può essere rapidamente fuso con il suo buddy adiacente per ricreare segmenti contigui di dimensione maggiore.
- **Inconveniente:** l'arrotondamento per eccesso genera frammentazione interna, dove lo spreco di memoria all'interno del segmento allocato è significativo (es. 33 KB usano 64 KB).

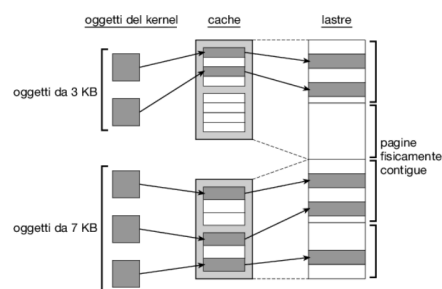


Allocazione a lastre

L'allocazione a lastre (slab allocation) è una strategia progettata per eliminare la frammentazione interna tipica del sistema buddy.

Strutture principali:

- **Lastra (Slab):** una o più pagine fisicamente contigue.
- **Cache:** consiste di una o più lastre. Esiste una cache dedicata per ogni categoria di struttura dati del kernel (es. una cache per i descrittori dei processi, una per i semafori).
- **Oggetti:** istanze delle strutture dati del kernel che popolano le lastre (es. un'istanza di `struct task_struct` in Linux).



Stati della lastra: Una lastra può essere:

- **Piena:** tutti gli oggetti sono usati.
- **Vuota:** tutti gli oggetti sono liberi.
- **Parzialmente occupata:** contiene oggetti sia usati che liberi.

Meccanismo di allocazione: l'allocatore cerca di soddisfare le richieste nell'ordine:

1. Oggetto libero da una lastra **parzialmente occupata**.
2. Oggetto libero da una lastra vuota.

3. Creazione e allocazione di una **nuova lastra** (da pagine continue) se le prime due opzioni falliscono.

Vantaggi principali:

1. **Annulla la frammentazione interna:** l'allocatore restituisce la quantità esatta di memoria necessaria per l'oggetto, poiché le lastre sono dimensionate con precisione per il tipo di oggetto che contengono.
2. **Velocità delle richieste:** le richieste di memoria sono soddisfatte rapidamente perché gli oggetti sono creati in anticipo (preallocati) nella cache. Quando un oggetto viene rilasciato, viene restituito alla cache e reso immediatamente disponibile.

Altre considerazioni

La progettazione dei sistemi di paginazione non si limita alla scelta dell'algoritmo di sostituzione e della politica di allocazione, ma deve includere altre considerazioni.

Prepaginazione

- **Problema:** un sistema di paginazione su richiesta ("puro") soffre di un alto numero di page fault iniziali all'avvio di un processo, dovuti al caricamento della sua località iniziale.
- **Obiettivo della prepaginazione:** prevenire questo elevato livello di paginazione iniziale, anticipando il caricamento delle pagine.
- **Meccanismo (es. working set):** se un processo viene sospeso (per I/O o mancanza di frame), il suo working set viene memorizzato. Quando il processo viene ripreso, l'intero working set viene riportato in memoria prima di riavviare l'esecuzione.
- **Costo-Beneficio:** la prepaginazione è conveniente solo se il costo delle eccezioni di page fault risparmiate è maggiore del costo della prepaginazione delle pagine non necessarie. Se una frazione α delle pagine prepaginate viene effettivamente usata, il parametro α deve essere prossimo a 1 affinché l'operazione sia vantaggiosa.

Dimensione delle pagine

La dimensione delle pagine è una scelta fondamentale nella progettazione di nuovi calcolatori, sebbene non esista una dimensione unica ottimale.

- **Pagine grandi (a favore):**
 - **Dimensione della tabella delle pagine:** diminuendo il numero di pagine, si riduce la dimensione della tabella delle pagine di ciascun processo.
 - **Tempo di I/O:** il tempo di trasferimento (proporzionale alla dimensione) è minimo rispetto al tempo di posizionamento e alla latenza. Pagine più grandi riducono l'I/O totale perché si ammortizzano i costi fissi di posizionamento e latenza su più dati, riducendo il tempo di I/O complessivo.
 - **Page fault:** pagine grandi riducono il numero totale di page fault (un page fault carica più dati).
- **Pagine piccole (a favore):**
 - **Frammentazione interna:** pagine piccole riducono la frammentazione interna (memoria sprecata nell'ultima pagina non completamente riempita).
 - **Località e I/O totale:** pagine piccole permettono una migliore risoluzione del working set e si adattano con maggiore precisione alla località, potenzialmente riducendo l'I/O totale (si caricano solo i dati realmente necessari).

Tendenza: storicamente la tendenza è verso l'incremento delle dimensioni delle pagine.

Portata del TLB (TLB Reach)

La portata del TLB è una metrica cruciale, definita come: **numero di elementi del TLB** $\times$ **dimensione delle pagine**. Idealmente, il TLB dovrebbe contenere l'intero working set del processo.

Strategie per aumentare la portata:

1. Aumentare il numero di elementi del TLB (costoso).
2. Aumentare la dimensione delle pagine (es. da 4 Kb a 16 Kb, quadruplicando la portata).
3. Utilizzare pagine di diverse dimensioni (huge pages in Linux), permettendo al SO di configurare pagine più grandi per le aree di memoria intensamente utilizzate.

Implicazioni: l'uso di diverse dimensioni di pagina richiede che la gestione del TLB sia svolta dal SO (non esclusivamente dall'HW), introducendo una penalizzazione prestazionale che è compensata dall'aumento del tasso di successi del TLB.

Tabella delle pagine invertita

La tabella delle pagine invertita riduce la memoria fisica necessaria per tenere traccia delle corrispondenze virtuale/fisica, utilizzando un elemento per ogni pagina fisica.

- **Necessità di tabelle esterne:** sebbene la tabella invertita gestisca l'allocazione, essa non contiene le informazioni complete sullo spazio degli indirizzi logici necessarie per il page fault (dove si trova la pagina nel backing store).
- **Gestione del page fault:** per supportare la paginazione su richiesta, è necessaria una tabella delle pagine esterna per ciascun processo, contenente le informazioni sulla locazione di ciascuna pagina virtuale.
- **Problema di efficienza:** le tabelle esterne sono consultate solo in caso di page fault, e sono esse stesse paginabili. Questo può causare un "doppio page fault" (un page fault che ne innesca un altro per caricare la tabella esterna, ritardando la ricerca della pagina).

Vincolo di I/O e vincolo delle pagine

In certi casi, è necessario che alcune pagine non possano essere selezionate come vittime e rimosse dalla memoria fisica.

- **Vincolo di I/O:** l'I/O verso o dalla memoria utente richiede che il buffer di destinazione sia **fisicamente presente** in memoria durante il trasferimento (pinning).
- **Problema dell'I/O:** se il processo I/O è messo in coda, un algoritmo di sostituzione globale potrebbe rimuovere il buffer dalla memoria (scaricandolo), e l'operazione I/O verrebbe eseguita su un frame ora occupato da un'altra pagina.
- **Soluzione (Bit di Vincolo):** a ogni frame si associa un bit di vincolo (lock bit). Se il bit è attivo, la pagina non può essere selezionata per la sostituzione. L'I/O vincola le pagine prima del trasferimento e rimuove il vincolo una volta completato.
- **Uso nella politica di gestione:** i bit di vincolo possono essere usati anche per evitare che una pagina appena caricata (es. da un processo a bassa priorità) venga immediatamente sostituita a favore di un processo ad alta priorità, sprecando il lavoro di I/O appena compiuto.
- **Abuso:** un uso eccessivo o errato dei bit di vincolo può rendere il frame permanentemente inutilizzabile.